```
import pandas as pd

# Load data
df = pd.read_csv("ncr_ride_bookings.csv")


# 1. REMOVE EXTRA


df['Booking ID'] = df['Booking ID'].str.replace('"', '')
df['Customer ID'] = df['Customer ID'].str.replace('"', '')

# 2. DATE & TIME FIX


df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df['Time'] = pd.to_datetime(df['Time'], errors='coerce').dt.time


# 3. NUMERIC CONVERSION


num_cols = [
    'Booking Value', 'Ride Distance',
    'Driver Ratings', 'Customer Rating',
    'Avg VTAT', 'Avg CTAT'
]

for col in num_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# 4. HANDLE MISSING VALUES


df['Booking Value'] = df['Booking Value'].fillna(0)



# Cancellation flags
df['Cancelled Rides by Customer'] = df['Cancelled Rides by Customer'].fillna(0)
df['Cancelled Rides by Driver'] = df['Cancelled Rides by Driver'].fillna(0)

# 5. STANDARDIZE TEXT


df['Booking Status'] = df['Booking Status'].str.strip()

# Optional: unify naming
df['Booking Status'] = df['Booking Status'].replace({
    'Cancelled by Customer': 'Cancelled',
    'Cancelled by Driver': 'Cancelled'
})


# 6. REMOVE DUPLICATES


df = df.drop_duplicates()


# 7. CREATE NEW COLUMNS

# Total Cancelled Flag
df['Is Cancelled'] = df['Booking Status'].apply(lambda x: 1 if 'Cancel' in str(x) else 0)

# Revenue column (safe)
df['Revenue'] = df['Booking Value']


# 8. FINAL CHECK


print(df.info())
print(df.isnull().sum())
```

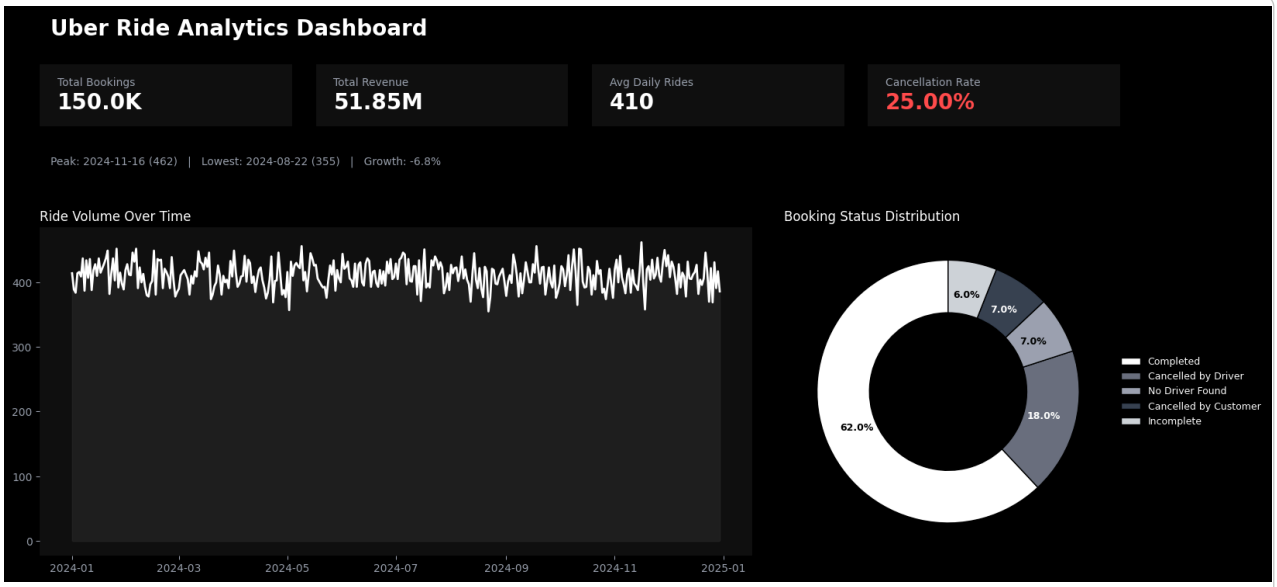
```
/tmp/ipykernel_19644/793856730.py:17: UserWarning: Could not infer format, so each element will be parsed individually, fa
  df['Time'] = pd.to_datetime(df['Time'], errors='coerce').dt.time
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150000 entries, 0 to 149999
Data columns (total 23 columns):
 #   Column                               Non-Null Count   Dtype
```

```
---  ------                                --------------   -----
 0   Date                                  150000 non-null  datetime64[ns]
 1   Time                                  150000 non-null  object
 2   Booking ID                            150000 non-null  object
 3   Booking Status                        150000 non-null  object
 4   Customer ID                           150000 non-null  object
 5   Vehicle Type                          150000 non-null  object
 6   Pickup Location                       150000 non-null  object
 7   Drop Location                         150000 non-null  object
 8   Avg VTAT                              139500 non-null  float64
 9   Avg CTAT                              102000 non-null  float64
 10  Cancelled Rides by Customer           150000 non-null  float64
 11  Reason for cancelling by Customer  10500 non-null   object
 12  Cancelled Rides by Driver             150000 non-null  float64
 13  Driver Cancellation Reason            27000 non-null   object
 14  Incomplete Rides                      9000 non-null    float64
 15  Incomplete Rides Reason               9000 non-null    object
 16  Booking Value                         150000 non-null  float64
 17  Ride Distance                         102000 non-null  float64
 18  Driver Ratings                        93000 non-null   float64
 19  Customer Rating                       93000 non-null   float64
 20  Payment Method                        102000 non-null  object
 21  Is Cancelled                          150000 non-null  int64
 22  Revenue                               150000 non-null  float64
dtypes: datetime64[ns](1), float64(10), int64(1), object(11)
memory usage: 26.3+ MB
None
Date                                        0
Time                                        0
Booking ID                                  0
Booking Status                              0
Customer ID                                 0
Vehicle Type                                0
Pickup Location                             0
Drop Location                               0
Avg VTAT                                10500
Avg CTAT                                48000
Cancelled Rides by Customer                 0
Reason for cancelling by Customer      139500
Cancelled Rides by Driver                   0
Driver Cancellation Reason             123000
Incomplete Rides                       141000
Incomplete Rides Reason                141000
Booking Value                               0
Ride Distance                           48000
Driver Ratings                          57000
Customer Rating                         57000
Payment Method                          48000
Is Cancelled                                0
Revenue                                     0
dtype: int64
```

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec

#load data
df = pd.read_csv("ncr_ride_bookings.csv")
df['Date'] = pd.to_datetime(df['Date'])


# METRICS

total_bookings = len(df)
total_revenue = df['Booking Value'].sum()

daily = df.groupby('Date').size()
avg_daily = daily.mean()

cancel_rate = (
    df[df['Booking Status'].str.contains("Cancel", na=False)].shape[0]
    / total_bookings
) * 100

status = df['Booking Status'].value_counts()


# INSIGHTS

peak_day, peak_val = daily.idxmax(), daily.max()
low_day, low_val = daily.idxmin(), daily.min()
growth = ((daily.iloc[-1] - daily.iloc[0]) / daily.iloc[0]) * 100


# FORMAT FUNCTION
```

```
def fmt(v):
    if v >= 1_000_000:
        return f"{v/1_000_000:.2f}M"
    elif v >= 1000:
        return f"{v/1000:.1f}K"
    return str(int(v))


# COLORS

bg = "#000000"
card = "#121212"
text = "#FFFFFF"
muted = "#9CA3AF"
bad = "#FF4D4F"

plt.style.use("dark_background")


fig = plt.figure(figsize=(18,10), facecolor=bg)

gs = GridSpec(
    nrows=8,
    ncols=12,
    figure=fig,
    height_ratios=[0.6, 1.2, 0.6, 0.2, 3, 3, 0.2, 0.1],
    hspace=0.35,
    wspace=0.35
)


ax_title = fig.add_subplot(gs[0, :])
ax_title.axis("off")
ax_title.text(
    0.01, 0.3,
    "Uber Ride Analytics Dashboard",
    fontsize=20,
    fontweight='bold',
    color=text
)


kpis = [
    ("Total Bookings", fmt(total_bookings), text),
    ("Total Revenue", fmt(total_revenue), text),
    ("Avg Daily Rides", fmt(avg_daily), text),
    ("Cancellation Rate", f"{cancel_rate:.2f}%", bad),
]

for i, (title, value, color) in enumerate(kpis):
    ax = fig.add_subplot(gs[1, i*3:(i+1)*3])
    ax.set_facecolor(card)

    for s in ax.spines.values():
        s.set_visible(False)

    ax.set_xticks([])
    ax.set_yticks([])

    ax.text(0.07, 0.65, title, fontsize=10, color=muted, transform=ax.transAxes)
    ax.text(
        0.07, 0.28, value,
        fontsize=20,
        fontweight='bold',
        color=color,
        transform=ax.transAxes
    )


ax_ins = fig.add_subplot(gs[2, :])
ax_ins.axis("off")

ax_ins.text(
    0.01, 0.5,
    f"Peak: {peak_day.date()} ({fmt(peak_val)})   |   "
    f"Lowest: {low_day.date()} ({fmt(low_val)})   |   "
    f"Growth: {growth:.1f}%",
    fontsize=10,
    color=muted,
    va='center'
)
```

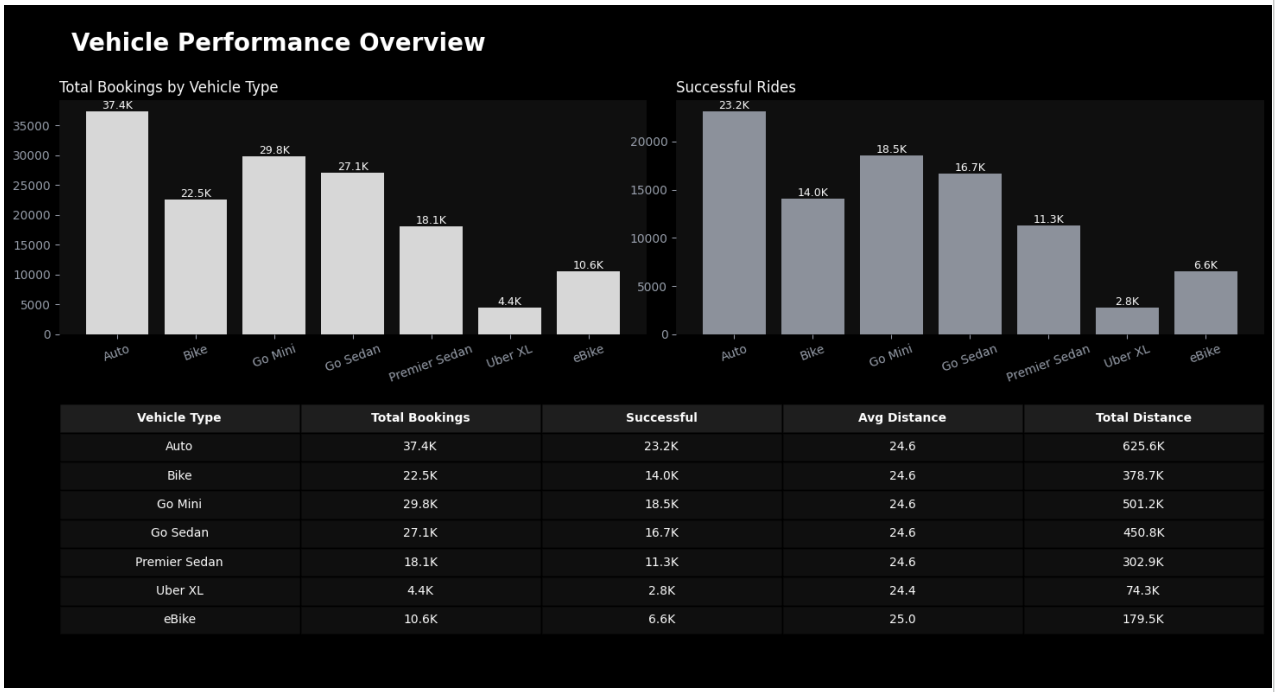
```
ax1 = fig.add_subplot(gs[4:6, :8])
ax1.set_facecolor(card)

ax1.plot(daily.index, daily.values, color="#FFFFFF", linewidth=2)
ax1.fill_between(daily.index, daily.values, color="#FFFFFF", alpha=0.05)

ax1.set_title("Ride Volume Over Time", fontsize=12, color=text, loc='left')

ax1.tick_params(colors=muted)

for s in ax1.spines.values():
    s.set_visible(False)


ax2 = fig.add_subplot(gs[4:6, 8:])
ax2.set_facecolor(card)

colors = ["#FFFFFF", "#6B7280", "#9CA3AF", "#374151", "#D1D5DB"]

def autopct_func(pct):
    return f"{pct:.1f}%"

wedges, texts, autotexts = ax2.pie(
    status.values,
    startangle=90,
    colors=colors[:len(status)],
    autopct=autopct_func,
    pctdistance=0.75,
    wedgeprops={
        'width': 0.4,
        'edgecolor': bg
    }
)

# remove labels
for t in texts:
    t.set_text("")

# smart contrast for %
for i, autotext in enumerate(autotexts):
    if colors[i] in ["#FFFFFF", "#D1D5DB", "#9CA3AF"]:
        autotext.set_color("#000000")
    else:
        autotext.set_color("#FFFFFF")

    autotext.set_fontsize(9)
    autotext.set_weight('bold')

# legend
ax2.legend(
    wedges,
    status.index,
    loc="center left",
    bbox_to_anchor=(1, 0.5),
    frameon=False,
    labelcolor=text,
    fontsize=9
)

ax2.set_title("Booking Status Distribution", fontsize=12, color=text, loc='left')

plt.show()
```




```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec


df = pd.read_csv("ncr_ride_bookings.csv")


df['Vehicle Type'] = df['Vehicle Type'].replace({
    'Mini': 'Go Mini',
    'Go': 'Go Mini',
    'Sedan': 'Go Sedan',
    'UberXL': 'Uber XL',
    'Premier': 'Premier Sedan'
})


vehicle_summary = df.groupby('Vehicle Type').agg(
    total_bookings=('Booking ID', 'count'),
    successful=('Booking Status', lambda x: (x == 'Completed').sum()),
    avg_distance=('Ride Distance', 'mean'),
    total_distance=('Ride Distance', 'sum')
).reset_index().fillna(0)


def format_k(x):
    return f"{x/1000:.1f}K" if x >= 1000 else f"{int(x)}"

vehicle_summary['Total Bookings'] = vehicle_summary['total_bookings'].apply(format_k)
vehicle_summary['Successful'] = vehicle_summary['successful'].apply(format_k)
vehicle_summary['Avg Distance'] = vehicle_summary['avg_distance'].round(1)
vehicle_summary['Total Distance'] = vehicle_summary['total_distance'].apply(format_k)

table_data = vehicle_summary[['Vehicle Type','Total Bookings','Successful','Avg Distance','Total Distance']]


bg = "#000000"
card = "#121212"
text = "#FFFFFF"
muted = "#9CA3AF"

plt.style.use("dark_background")


fig = plt.figure(figsize=(18,10), facecolor=bg)

gs = GridSpec(
```

```
    8, 12,
    figure=fig,
    hspace=0.4,
    wspace=0.4
)


ax_title = fig.add_subplot(gs[0, :])
ax_title.axis("off")
ax_title.text(0.01, 0.4, "Vehicle Performance Overview",
              fontsize=20, fontweight='bold', color=text)


ax1 = fig.add_subplot(gs[1:4, :6])
ax1.set_facecolor(card)

bars = ax1.bar(
    vehicle_summary['Vehicle Type'],
    vehicle_summary['total_bookings'],
    color="#FFFFFF",
    alpha=0.85
)

for bar, val in zip(bars, vehicle_summary['Total Bookings']):
    ax1.text(
        bar.get_x()+bar.get_width()/2,
        bar.get_height(),
        val,
        ha='center',
        va='bottom',
        color=text,
        fontsize=9
    )

ax1.set_title("Total Bookings by Vehicle Type", color=text, loc='left')
ax1.tick_params(axis='x', rotation=20, colors=muted)
ax1.tick_params(axis='y', colors=muted)

for s in ax1.spines.values():
    s.set_visible(False)


ax2 = fig.add_subplot(gs[1:4, 6:])
ax2.set_facecolor(card)

bars = ax2.bar(
    vehicle_summary['Vehicle Type'],
    vehicle_summary['successful'],
    color="#9CA3AF",
    alpha=0.9
)

for bar, val in zip(bars, vehicle_summary['Successful']):
    ax2.text(
        bar.get_x()+bar.get_width()/2,
        bar.get_height(),
        val,
        ha='center',
        va='bottom',
        color=text,
        fontsize=9
    )

ax2.set_title("Successful Rides", color=text, loc='left')
ax2.tick_params(axis='x', rotation=20, colors=muted)
ax2.tick_params(axis='y', colors=muted)

for s in ax2.spines.values():
    s.set_visible(False)


ax3 = fig.add_subplot(gs[4:, :])
ax3.axis('off')
ax3.set_facecolor(card)

table = ax3.table(
    cellText=table_data.values,
    colLabels=table_data.columns,
    loc='center',
    cellLoc='center'
)
```

```
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 2)

# STYLE TABLE
for (row, col), cell in table.get_celld().items():
    cell.set_edgecolor(bg)

    if row == 0:
        cell.set_facecolor("#1F1F1F")
        cell.set_text_props(color=text, weight='bold')
    else:
        cell.set_facecolor(card)
        cell.set_text_props(color=text)


plt.show()
```

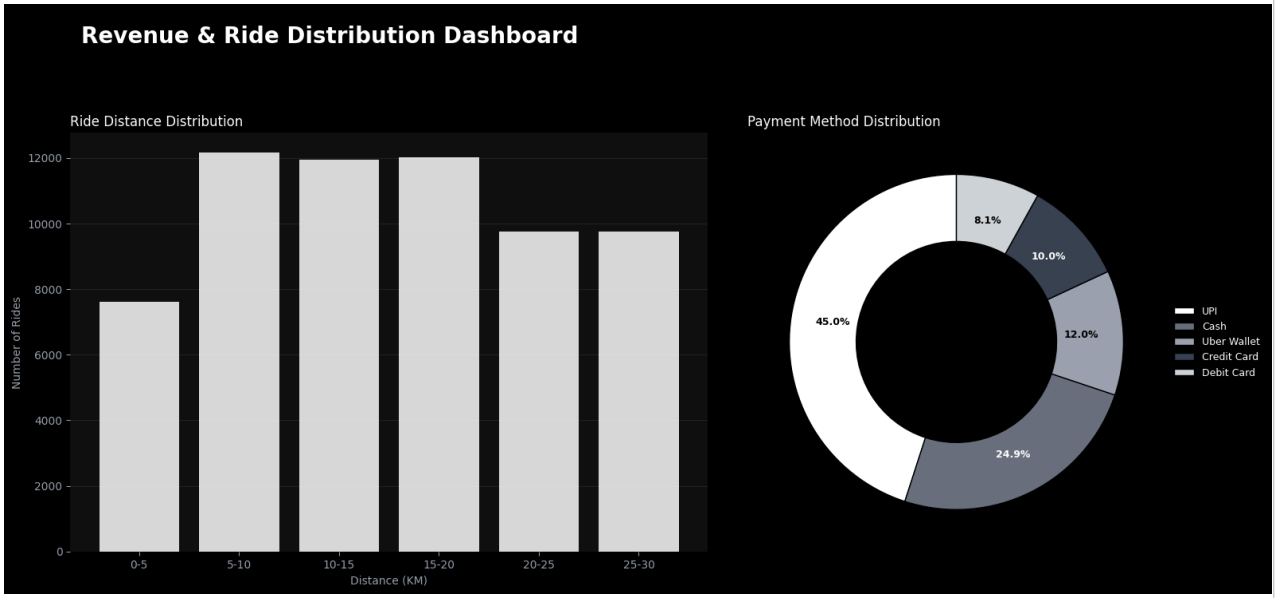


Vehicle Type	Total Bookings	Successful	Avg Distance	Total Distance
Auto	37.4K	23.2K	24.6	625.6K
Bike	22.5K	14.0K	24.6	378.7K
Go Mini	29.8K	18.5K	24.6	501.2K
Go Sedan	27.1K	16.7K	24.6	450.8K
Premier Sedan	18.1K	11.3K	24.6	302.9K
Uber XL	4.4K	2.8K	24.4	74.3K
eBike	10.6K	6.6K	25.0	179.5K

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec


df = pd.read_csv("ncr_ride_bookings.csv")


df['Ride Distance'] = pd.to_numeric(df['Ride Distance'], errors='coerce')

bins = [0, 5, 10, 15, 20, 25, 30]
labels = ["0-5", "5-10", "10-15", "15-20", "20-25", "25-30"]

df['Distance Bin'] = pd.cut(df['Ride Distance'], bins=bins, labels=labels)
distance_counts = df['Distance Bin'].value_counts().sort_index()

payment_counts = df['Payment Method'].value_counts()


bg = "#000000"
card = "#121212"
text = "#FFFFFF"
muted = "#9CA3AF"

plt.style.use("dark_background")


fig = plt.figure(figsize=(18,10), facecolor=bg)

gs = GridSpec(
    nrows=6,
    ncols=12,
```

```
    figure=fig,
    height_ratios=[0.8, 0.2, 3, 3, 0.2, 0.1],
    hspace=0.35,
    wspace=0.35
)


ax_title = fig.add_subplot(gs[0, :])
ax_title.axis("off")

ax_title.text(
    0.01, 0.4,
    "Revenue & Ride Distribution Dashboard",
    fontsize=20,
    fontweight='bold',
    color=text
)


ax1 = fig.add_subplot(gs[2:4, :7])
ax1.set_facecolor(card)


bars = ax1.bar(
    distance_counts.index.astype(str),
    distance_counts.values,
    color="#FFFFFF",
    alpha=0.85
)

ax1.set_title("Ride Distance Distribution", fontsize=12, color=text, loc='left')
ax1.set_xlabel("Distance (KM)", color=muted)
ax1.set_ylabel("Number of Rides", color=muted)

ax1.tick_params(colors=muted)
ax1.grid(axis='y', alpha=0.08)

for s in ax1.spines.values():
    s.set_visible(False)


ax2 = fig.add_subplot(gs[2:4, 7:])
ax2.set_facecolor(card)

colors = ["#FFFFFF", "#6B7280", "#9CA3AF", "#374151", "#D1D5DB"]

def autopct_func(pct):
    return f"{pct:.1f}%"

wedges, texts, autotexts = ax2.pie(
    payment_counts.values,
    startangle=90,
    colors=colors[:len(payment_counts)],
    autopct=autopct_func,
    pctdistance=0.75,
    wedgeprops={
        'width': 0.4,
        'edgecolor': bg
    }
)

# REMOVE LABEL OVERLAP
for t in texts:
    t.set_text("")

# SMART TEXT COLOR
for i, autotext in enumerate(autotexts):
    if colors[i] in ["#FFFFFF", "#D1D5DB", "#9CA3AF"]:
        autotext.set_color("#000000")
    else:
        autotext.set_color("#FFFFFF")

    autotext.set_fontsize(9)
    autotext.set_weight('bold')

# LEGEND
ax2.legend(
    wedges,
    payment_counts.index,
    loc="center left",
    bbox_to_anchor=(1, 0.5),
    frameon=False,
```

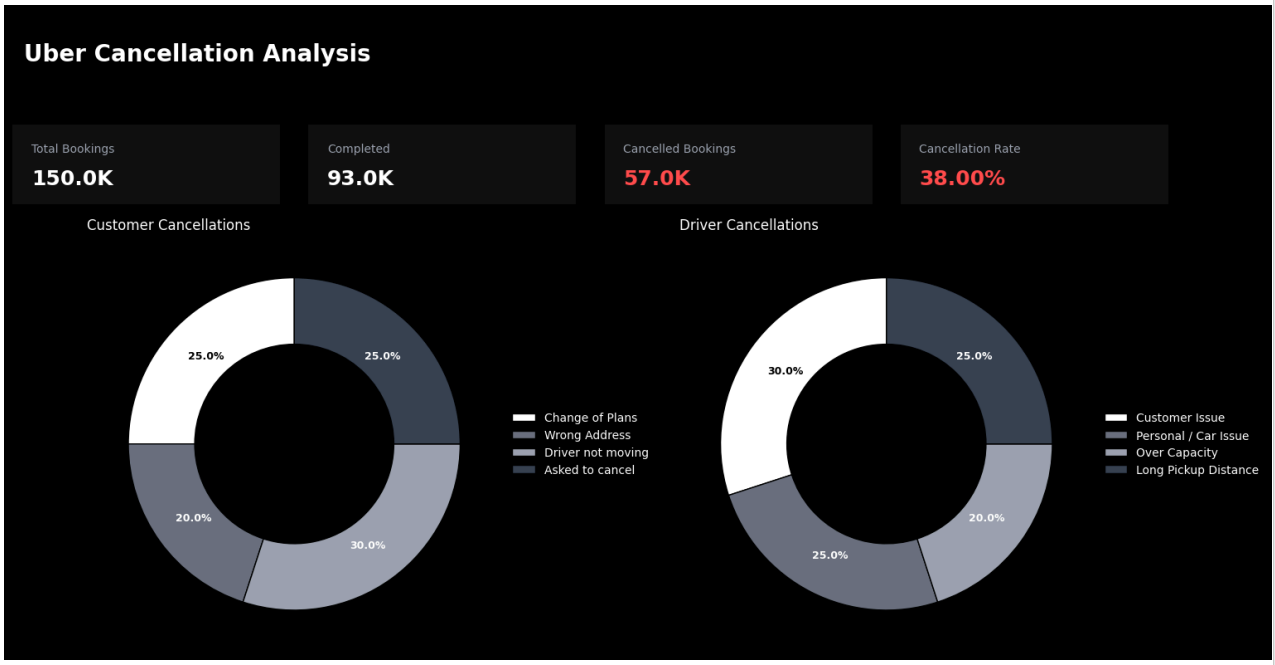
```
    labelcolor=text,
    fontsize=9
)

ax2.set_title("Payment Method Distribution", fontsize=12, color=text, loc='left')


plt.show()
```




```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec

df = pd.read_csv("ncr_ride_bookings.csv")

df['Booking Value'] = pd.to_numeric(df['Booking Value'], errors='coerce')
df['Ride Distance'] = pd.to_numeric(df['Ride Distance'], errors='coerce')


total_bookings = len(df)
success_bookings = df[df['Booking Status'] == 'Completed'].shape[0]
cancelled_bookings = total_bookings - success_bookings
cancellation_rate = (cancelled_bookings / total_bookings) * 100


# FORMAT

def format_k(x):
    return f"{x/1000:.1f}K" if x >= 1000 else str(int(x))


# UBER COLORS
bg = "#000000"
card = "#121212"
text = "#FFFFFF"
muted = "#9CA3AF"
red = "#FF4D4F"

plt.style.use("dark_background")


fig = plt.figure(figsize=(18,10), facecolor=bg)

gs = GridSpec(
    6, 12,
    figure=fig,
    hspace=0.4,
```

```
    wspace=0.4
)


ax_title = fig.add_subplot(gs[0, :])
ax_title.axis("off")

ax_title.text(
    0.01, 0.4,
    "Uber Cancellation Analysis",
    fontsize=20,
    fontweight="bold",
    color=text
)


kpis = [
    ("Total Bookings", format_k(total_bookings), text),
    ("Completed", format_k(success_bookings), text),
    ("Cancelled", format_k(cancelled_bookings), red),
    ("Cancellation Rate", f"{cancellation_rate:.2f}%", red),
]

for i, (title, value, color) in enumerate(kpis):
    ax = fig.add_subplot(gs[1, i*3:(i+1)*3])
    ax.set_facecolor(card)

    for s in ax.spines.values():
        s.set_visible(False)

    ax.set_xticks([])
    ax.set_yticks([])

    ax.text(0.07, 0.65, title, fontsize=10, color=muted, transform=ax.transAxes)
    ax.text(0.07, 0.25, value, fontsize=18, fontweight='bold', color=color, transform=ax.transAxes)


ax1 = fig.add_subplot(gs[2:, :6])
ax1.set_facecolor(card)

customer_reasons = pd.Series({
    "Change of Plans": 25,
    "Wrong Address": 20,
    "Driver not moving": 30,
    "Asked to cancel": 25
})

colors = ["#FFFFFF", "#6B7280", "#9CA3AF", "#374151"]

def autopct(pct):
    return f"{pct:.1f}%"

wedges, texts, autotexts = ax1.pie(
    customer_reasons.values,
    startangle=90,
    colors=colors,
    autopct=autopct,
    pctdistance=0.75,
    wedgeprops=dict(width=0.4, edgecolor=bg)
)

# remove labels
for t in texts:
    t.set_text("")

# smart contrast
for i, t in enumerate(autotexts):
    t.set_color("#000000" if colors[i] in ["#FFFFFF", "#D1D5DB"] else "#FFFFFF")
    t.set_fontsize(9)
    t.set_weight('bold')

ax1.legend(
    wedges,
    customer_reasons.index,
    loc="center left",
    bbox_to_anchor=(1, 0.5),
    frameon=False,
    labelcolor=text
)

ax1.set_title("Customer Cancellations", color=text, loc='left')
```

```
ax2 = fig.add_subplot(gs[2:, 6:])
ax2.set_facecolor(card)

driver_reasons = pd.Series({
    "Customer Issue": 30,
    "Personal / Car Issue": 25,
    "Over Capacity": 20,
    "Long Pickup Distance": 25
})

wedges, texts, autotexts = ax2.pie(
    driver_reasons.values,
    startangle=90,
    colors=colors,
    autopct=autopct,
    pctdistance=0.75,
    wedgeprops=dict(width=0.4, edgecolor=bg)
)

for t in texts:
    t.set_text("")

for i, t in enumerate(autotexts):
    t.set_color("#000000" if colors[i] in ["#FFFFFF", "#D1D5DB"] else "#FFFFFF")
    t.set_fontsize(9)
    t.set_weight('bold')

ax2.legend(
    wedges,
    driver_reasons.index,
    loc="center left",
    bbox_to_anchor=(1, 0.5),
    frameon=False,
    labelcolor=text
)

ax2.set_title("Driver Cancellations", color=text, loc='left')


plt.show()
```




```
import pandas as pd

# Data
data = {
    'Vehicle': ['Go Mini', 'Go Sedan', 'UberXL', 'Premier Sedan', 'Auto', 'Bike', 'eBike'],
    'Customer Rating': [4.40, 4.41, 4.40, 4.40, 4.40, 4.40, 4.40],
    'Driver Rating': [4.23, 4.23, 4.24, 4.23, 4.23, 4.23, 4.23],
    'Revenue': [10338496, 9369719, 1528032, 6275332, 12878422, 7837697, 3618485]
}
```

```
df = pd.DataFrame(data)

styled_df = df.style \
    .format({
        'Customer Rating': '{:.2f}',
        'Driver Rating': '{:.2f}',
        'Revenue': 'Rs {:,}'
    }) \
    .set_properties(**{
        'background-color': '#000000',
        'color': 'white',
        'text-align': 'center',
        'font-size': '14px',
        'border': '1px solid #222'
    }) \
    .set_table_styles([
        {
            'selector': 'th',
            'props': [
                ('background-color', '#000000'),
                ('color', '#9ca3af'),
                ('font-weight', '600'),
                ('border-bottom', '1px solid #333'),
                ('text-align', 'center'),
                ('font-size', '13px')
            ]
        },
        {
            'selector': 'td',
            'props': [
                ('border', '1px solid #222')
            ]
        }
    ]) \
    .highlight_max(subset=['Customer Rating'], color='#16a34a') \
    .highlight_max(subset=['Driver Rating'], color='#16a34a') \
    .highlight_max(subset=['Revenue'], color='#065f46')

styled_df
```

	Vehicle	Customer Rating	Driver Rating	Revenue
0	Go Mini	4.40	4.23	Rs 10,338,496
1	Go Sedan	4.41	4.23	Rs 9,369,719
2	UberXL	4.40	4.24	Rs 1,528,032
3	Premier Sedan	4.40	4.23	Rs 6,275,332
4	Auto	4.40	4.23	Rs 12,878,422
5	Bike	4.40	4.23	Rs 7,837,697
6	eBike	4.40	4.23	Rs 3,618,485